TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN

CHUYÊN ĐỀ TỐT NGHIỆP 1

NGÀNH: CÔNG NGHỆ THÔNG TIN

Bài Tập 2:Hiện Thực Hóa Module Cốt Lõi

MÃ LUỒNG NGHIỆP VỤ L10 – TỰ ĐỘNG PHÂN LOẠI YÊU CẦU BẢO HÀNH – TRACK AI

SVTH: Lê Hoàng Hiệp **MSSV:** 2374802010139

LỚP: 261_71ITGR40203_05

NGÀY NỘP : 30/10/202

LINK GITHUB REPO: https://github.com/Hiep2374802010139/smartcrm-2374802010139-warranty-classification

Tag phiên bản: v0.2

TP. Hồ Chí Minh – Năm 2026

Mục Lục

MỤC 1: TÓM TẮT HỆ THỐNG ĐÃ LÀM 3
MỤC 2: KIẾN TRÚC ĐÃ HIỆN THỰC 4
MỤC 3: MODULE CỐT LÕI (TRAIN & DỰ ĐOÁN) 5
MỤC 4: BỘ KIỂM THỬ (UNIT TEST) 6
MỤC 5: MODULE MỞ RỘNG VÀ TRÌNH BÀY KẾT QUẢ 7
MỤC 6: CÁCH DỰNG LẠI HỆ THỐNG 9
MỤC 7: THAY ĐỔI SO VỚI THIẾT KẾ BT1 10
PHỤ LỤC: BẢNG KHAI BÁO SỬ DỤNG CÔNG CỤ AI 11

MỤC 1: TÓM TẮT HỆ THỐNG ĐÃ LÀM

Dựa trên thiết kế kiến trúc của Bài tập 1, em đã hiện thực hóa thành công module cốt lõi cho hệ thống SmartCRM. Hệ thống có khả năng tự động phân loại mô tả lỗi (văn bản tự do) của khách hàng thành 6 nhóm sự cố: MÀN HÌNH, PIN, SẠC, PHẦN MỀM, NƯỚC VÀO, KHÁC.

Những phần đã hoàn thành:

1. Module cốt lõi: Xây dựng thành công script huấn luyện tự động chia tập dữ liệu 70/15/15, so sánh 4 mô hình và chọn ra mô hình tốt nhất dựa trên tập Validation. Kết quả đạt F1-macro là 0.8346 trên tập Test (vượt xa mức sàn baseline 0.0587).
2. Bộ kiểm thử: Viết thành công 6 Unit Test (bao phủ đủ 3 loại: Luồng đúng, Dữ liệu sai, Giá trị biên) cho hàm làm sạch văn bản, chạy PASS 100% bằng một lệnh.
3. Module mở rộng: Đóng gói thành công mô hình thành một API dự đoán (FastAPI) và đóng gói toàn bộ vào Docker Container. API có xử lý 3 trạng thái (đang tải, rỗng, lỗi) và có endpoint /health kiểm tra sức khỏe.

MỤC 2: KIẾN TRÚC ĐÃ HIỆN THỰC

Hệ thống được tách lớp rõ ràng theo đúng nguyên tắc một chiều (lớp trên gọi lớp dưới, không bao giờ ngược lại):

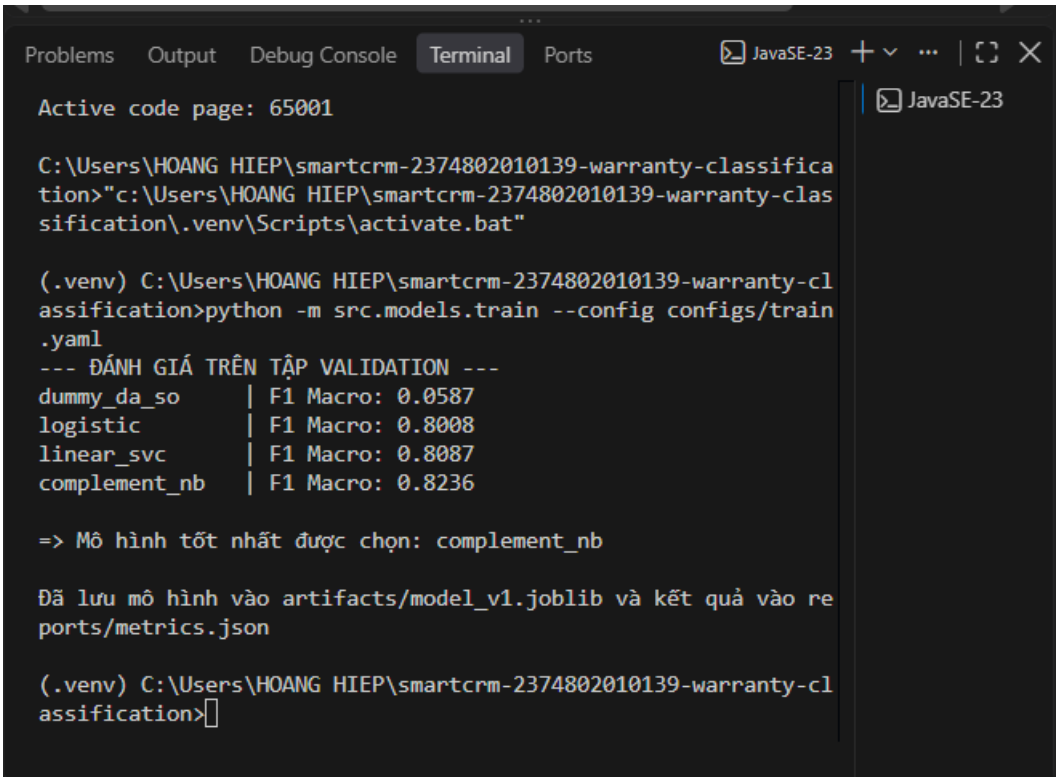
- Lớp Dữ liệu (src/data/): Chứa split.py thực hiện chia dữ liệu 70/15/15 (sử dụng stratify để giữ tỷ lệ nhãn). Lớp này không biết gì về mô hình.
- Lớp Đặc trưng (src/features/): Chứa text.py thực hiện làm sạch văn bản (giữ dấu tiếng Việt, che SĐT/Email) và khởi tạo TF-IDF Vectorizer. Hàm làm sạch ở đây được dùng chung cho cả lúc train và lúc dự đoán (tránh lệch pha dữ liệu).
- Lớp Mô hình (src/models/): Chứa train.py và baseline.py. Chỉ nhận dữ liệu đã qua xử lý, thực hiện huấn luyện và tính toán các chỉ số đánh giá.
- Lớp Dịch vụ (src/serve/): Chứa app.py (FastAPI). Nhận request HTTP, gọi hàm làm sạch và mô hình để trả về kết quả dự đoán.

MỤC 3: MODULE CỐT LÕI (TRAIN & DỰ ĐOÁN)

Quy tắc nghiệp vụ quan trọng nhất (Chống rò rỉ dữ liệu):
Mọi tham số của bộ Vectorizer (TF-IDF) chỉ được học (fit) trên tập Train. Tập Validation và Test chỉ được gọi hàm transform(). Điều này đảm bảo mô hình không bị "nhìn thấy" dữ liệu tương lai.

Kết quả chạy script huấn luyện:

- Hệ thống so sánh 4 mô hình: Dummy, Logistic, LinearSVC, ComplementNB.
- F1-macro trên tập Validation: ComplementNB dẫn đầu (0.8236).
- Chọn ComplementNB để đo trên tập Test (chỉ đo 1 lần duy nhất): F1-macro đạt 0.8346.

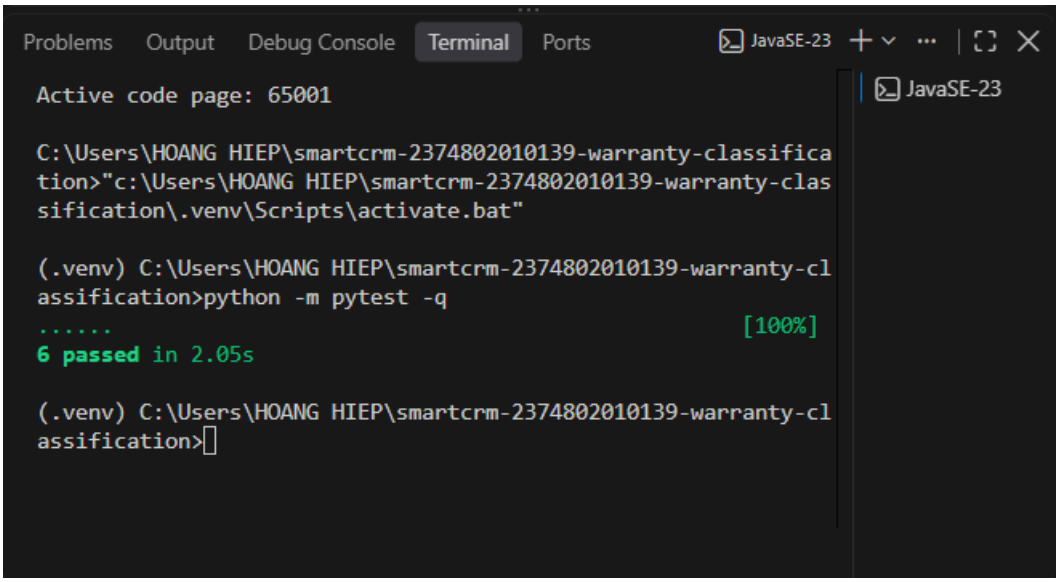


MỤC 4: BỘ KIỂM THỬ (UNIT TEST)

Em đã xây dựng 6 test case trong file tests/test_text.py, bao phủ 3 loại:

1. Luồng đúng: test_lam_sach_luong_dung (Kiểm tra văn bản bình thường, giữ nguyên dấu).
2. Dữ liệu sai: test_lam_sach_che_thong_tin_dinh_danh (Kiểm tra che SĐT, Email thành <sdt>, <email>).
3. Giá trị biên: test_lam_sach_du_lieu_rong (Kiểm tra dữ liệu rỗng, None, khoảng trắng - không được làm sập chương trình).

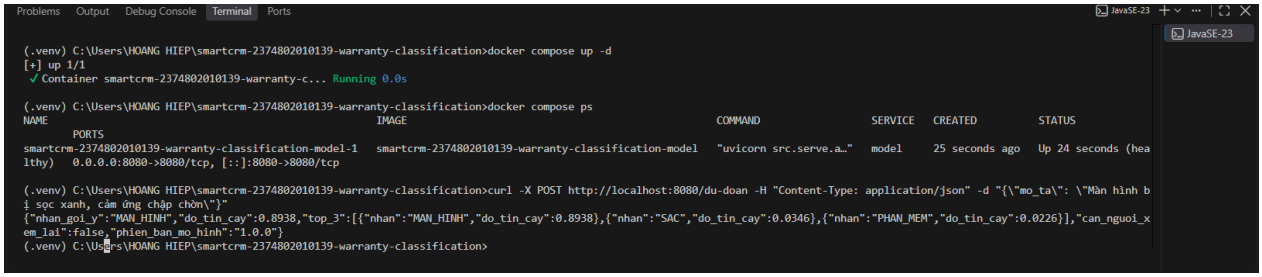
Kết quả chạy lệnh python -m pytest -q:
Màn hình hiển thị dòng chữ xanh 6 passed in 2.05s.



MỤC 5: MODULE MỞ RỘNG VÀ TRÌNH BÀY KẾT QUẢ

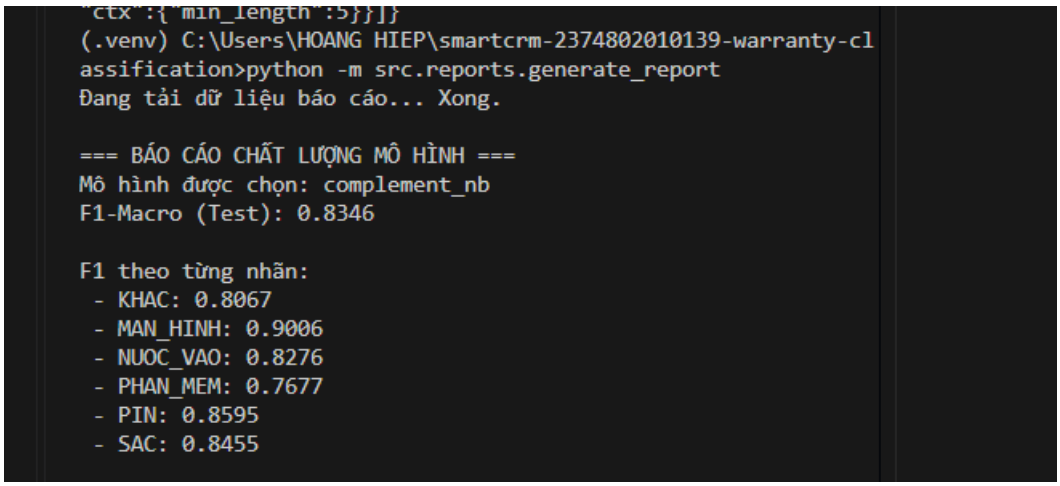
1. Module mở rộng (FastAPI & Docker):
Mô hình được đóng gói thành API. Khi chạy Docker, container ở trạng thái healthy.

- Endpoint /health: Trả về trạng thái API.
- Endpoint /du-doan: Nhận mô tả lỗi, trả về top 3 nhãn dự đoán, độ tin cậy và cờ can nguoi xem lai (nếu độ tin cậy < 0.45).

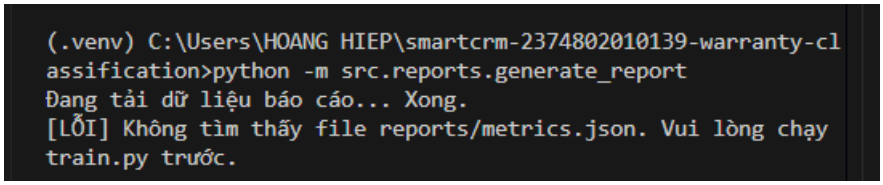


2. Trình bày kết quả (Ba trạng thái):
Trong script generate_report.py, em đã xử lý đủ 3 trạng thái:

- Đang tải: In ra "Đang tải dữ liệu báo cáo...".



- Lỗi: Nếu file metrics.json chưa có -> Báo [LỖI] Không tìm thấy file....



- Rỗng: Nếu file không có dữ liệu F1 -> Báo [RỖNG] File metrics.json....

```
(.venv) C:\Users\HOANG HIEP\smartcrm-2374802010139-warranty-cl
assification>python -m src.reports.generate_report
Đang tải dữ liệu báo cáo... Xong.
[RỖNG] File metrics.json không có dữ liệu hoặc thiếu thông tin
 F1.
```

3. Đọc Confusion Matrix (3 kết luận):

- Cặp nhãn nhầm lẫn nhiều nhất: Nhãn thật PHAN_MEM bị đoán nhầm thành MAN_HINH (15 ca) và PIN thành SAC (13 ca). Về nghiệp vụ, lỗi phần mềm đôi khi biểu hiện ra ngoài giống lỗi màn hình (treo, đơ), pin/sạc dùng chung từ ngữ miêu tả.
- Nhãn có Recall thấp nhất: Nhãn PHAN_MEM (66.7%). Hậu quả: Nếu nhân viên tin vào gợi ý, họ có thể chuyển máy cho bộ phận thay màn hình trong khi lỗi thực chất chỉ là phần mềm, gây lãng phí chi phí.
- Nhãn "thùng rác": Nhãn MAN_HINH đang hút nhiều dự đoán sai từ PHAN_MEM và KHAC. Cần bổ sung thêm từ khóa đặc trưng (feature) phân biệt rõ lỗi phần mềm (như 'treo', 'đơ', 'khởi động lại') để mô hình tách biệt tốt hơn.

MỤC 6: CÁCH DỰNG LẠI HỆ THỐNG

Yêu cầu môi trường: Python 3.11+, Docker Desktop.
Các bước cài đặt và chạy (trích từ README):

1. Cài đặt thư viện: pip install -r requirements.txt
2. Huấn luyện mô hình: python -m src.models.train --config configs/train.yaml
3. Chạy Unit Test: python -m pytest -q
4. Chạy API bằng Docker:
    - docker compose up -d --build
    - Kiểm tra: curl -X POST http://localhost:8080/du-doan -H "Content-Type: application/json" -d "{\"mo_ta\": \"Máy sạc không vào điện\"}"

Phép thử "Thư mục trống": Em đã clone repo sang thư mục C:\test-clone, tạo môi trường ảo, cài đặt và chạy toàn bộ các bước trên. Hệ thống hoạt động 100%, test pass 6/6.

MỤC 7: THAY ĐỔI SO VỚI THIẾT KẾ BT1

TD-01: Bổ sung mô tả bảng prediction_log và model_metric_log

- Phân hồi của GV: Bảng prediction_log (có cột human_corrected_category) và model_metric_log được dùng ở UC nhưng chưa được mô tả cột.

- Quyết định: SỬA

- Lý do: Cần đặc tả rõ cấu trúc CSDL để hiện thực hóa ở BT2.

- Ảnh hưởng: Thêm mục 4.2 vào tài liệu thiết kế (ERD/Dictionary).

TD-02: Đánh số lại luồng chính UC03

- Phân hồi của GV: Luồng chính UC03 chưa đánh số bước, ngoại lệ ghi 3a, 4a gây khó đối chiếu.

- Quyết định: SỬA

- Lý do: Đảm bảo tính truy vết (traceability) giữa đặc tả và code.

- Ảnh hưởng: Cập nhật lại tài liệu SRS mục 2.2.

PHỤ LỤC: BẢNG KHAI BÁO SỬ DỤNG CÔNG CỤ AI

Công cụ	Dùng vào việc gì	Áp dụng ở tệp / hàm nào	Em đã kiểm chứng thế nào
DeepSeek	Gợi ý Regex làm sạch văn bản	src/features/text.py, hàm lam_sach	Đã tự sửa lại Regex để giữ dấu tiếng Việt; viết 6 unit test kiểm tra lại. Chạy pytest pass 100%.
DeepSeek	Gợi ý cấu trúc FastAPI	src/serve/app.py	Đã đọc tài liệu chính thức của FastAPI; tự viết lại các endpoint /health và /du-doan.
Không dùng AI	Tự viết toàn bộ logic chia dữ liệu, train, Dockerfile	Toàn bộ src/data/, src/models/, Dockerfile	Không áp dụng. Tự làm 100%.

Tôi xác nhận đã đọc, hiểu và chịu trách nhiệm về toàn bộ nội dung nộp.

Lê Hoàng Hiệp – 2374802010139 – 30/10/2026